



Secure Cloud-Edge Deployments, with Trust

Stefano Forti^{*}, Gian-Luigi Ferrari, Antonio Brogi

Department of Computer Science, University of Pisa, Italy

ARTICLE INFO

Article history:

Received 15 January 2019

Received in revised form 27 June 2019

Accepted 27 August 2019

Available online 2 September 2019

Keywords:

Secure application deployment

Declarative programming

Probabilistic reasoning

ABSTRACT

Assessing the security level of IoT applications to be deployed to heterogeneous Cloud-Edge infrastructures operated by different providers is a non-trivial task. In this article, we present a methodology that permits to express security requirements for IoT applications, as well as infrastructure security capabilities, in a simple and declarative manner, and to automatically obtain an explainable assessment of the security level of the possible application deployments. The methodology also considers the impact of trust relations among different stakeholders using or managing Cloud-Edge infrastructures. A lifelike example is used to showcase the prototyped implementation of the methodology.

© 2019 Elsevier B.V. All rights reserved.

1. Introduction microservizi?

Enforcing Quality-of-Service (QoS) requirements in the deployment of Internet-of-Things (IoT) software systems is a non-trivial – yet necessary – task to accomplish. Indeed, such systems are often developed in large, highly distributed, multi-service architectures, which are to be deployed to complex, heterogeneous and highly distributed infrastructures, spanning the Cloud-IoT continuum. Cloud-Edge computing [1] extends Cloud computing towards the edge of the Internet to better support latency-sensitive and bandwidth-hungry IoT applications by mapping service application functionalities (e.g., device management, telemetry ingestion, processing and storage, status and notification, multi-level analytics and data visualisation [2]) wherever it is “best-placed” to suitably meet all the application (hardware, software and QoS) requirements [3,4].

Various works (e.g., [5–11]) have tackled the problem of determining optimal placements (and management) of application services, mainly taking into account resource usage, deployment costs, network QoS (i.e., latency, response time, bandwidth), and energy consumption. Considering these aspects all together is of primary importance in Cloud-Edge scenarios, where many life-critical (e.g., e-health, autonomous vehicles) or mission-critical (e.g., drone packet deliveries, smart farming) application verticals can significantly suffer from performance degradation due to bad resource allocation or insufficient Internet connectivity. Analogously, their management might aim at reducing operational costs, due to power consumption or to resource leasing, so to increase profit.

However, to the best of our knowledge no approach has been proposed that accounts for the security requirements of the application to be deployed and matches them to the security capabilities available at different infrastructure nodes. Security of ICT solutions is an intrinsically complex problem, which requires reasoning about a system model by analysing its security properties and their effectiveness against potential attacks, and accounting for trust relations among different involved stakeholders (e.g., infrastructure and application operators). In addition, any methodology for optimal service placement that accounts for security, should in principle enable decision-makers to understand why a certain deployment can be considered optimal, i.e. the provided recommendations should be explainable. Indeed, explainable artificial intelligence (XAI) techniques are getting more attention from the security community since they can provide a concise explanation (proof) of the query results [12]. In the case of determining secure and trustworthy deployments of an application, for instance, XAI aims at answering questions like: *Why is this deployment more secure than this other? Why and how much are they secure?*

As an extension to the Cloud, Cloud-Edge will share with it many security threats, while including its new peculiar ones. On the one hand, Cloud-Edge will increase the number of security enforcement points by allowing local processing of private data closer to the IoT sources. On the other hand, new infrastructures will have to face brand new threats for what concerns the physical vulnerability of devices. Indeed, application deployments to Cloud-Edge infrastructures will often include accessible (Edge or IoT) devices that may be easily hacked, broken or even stolen by malicious users and that can only offer a limited set of security capabilities [13,14].

Last but not least, as Cloud-Edge application deployments will likely span various service providers (some of which may be not trustworthy), trust must be considered when deciding where to

^{*} Corresponding author.

E-mail address: stefano.forti@di.unipi.it (S. Forti).

place application services. Trust relations are especially important at the edge of the Internet, due to the uncertainty derived from dealing with multiple infrastructure providers. Indeed, the security levels and reputation of Edge computing capabilities might be as heterogeneous as their hardware capabilities, and might give rise to security zones, i.e. collections of assets that share the exposure to certain security risks [15]. Thus, in Cloud-Edge scenarios, where part of the application could be deployed to a mixture of well-established (e.g., Clouds, ISPs) and opportunistic infrastructures (e.g., crowd-computing, ad-hoc networks [16]), trust models are needed to estimate trust levels towards unknown providers, possibly aggregating also trusted providers' opinions.

All this considered, the move of utility computing towards the edge of the network – and in continuity with existing Clouds – calls for new quantitative and explainable methodologies that permit to assess the security level of distributed multi-service IoT applications. Such methodologies should take into account application requirements, security countermeasures featured by the Cloud-Edge infrastructure, and trust relations in place among different stakeholders that manage or use Cloud-Edge infrastructures.

In this paper, we propose a first step towards well-founded and declarative reasoning methodologies for security- and trust-aware multi-service application deployment in Cloud-Edge scenarios. Our proposal, SecFog, helps application operators in Cloud-Edge scenarios in determining the most secure application deployments by reducing manual tuning and by considering specific application requirements, infrastructure capabilities and trust. SecFog has been prototyped in the ProbLog language [17] and, as we will show, the prototype can be used together with existing approaches that solve the problem of mapping IoT application services to Cloud-Edge infrastructures according to requirements other than security and trust (e.g., hardware, network QoS, cost). A generalised algebraic extension of the prototype, α SecFog, is also presented. Such an extension features the possibility of using different (semiring-based) trust models so to weight (and assess) the security level of eligible application deployments.

The rest of this paper is organised as follows. After briefly introducing the ProbLog language (Section 2), needed to understand the proposed solution, we detail (Section 3) the SecFog methodology and its implementation, while showing how it can be used on simple examples, relying on a simple default probabilistic trust model. Afterwards, we describe a larger lifelike example of secure application deployment and illustrate how the SecFog prototype can be used along with other existing tools for application placement in Cloud-Edge scenarios (Section 4). Then, in Section 5, we discuss how SecFog can embed more complex trust models based on the default one, and we present an algebraic extension of the basic prototype, α SecFog, that permits relying on arbitrary semiring-based trust models. Finally, after discussing related work (Section 6), we draw some conclusions and point to some directions for future work (Section 7).

2. Background: The ProbLog language

Being SecFog a declarative methodology based on probabilistic reasoning about declared infrastructure capabilities and security requirements, it was natural to prototype it by relying on probabilistic logic programming. To implement both the model and the matching strategy we used a language called ProbLog [18]. ProbLog is a Python package that permits writing logic programs that encode complex interactions between sets of heterogeneous components, capturing the inherent uncertainties that are present in real-life situations.

ProbLog programs are logic programs in which some of the facts are annotated with (their) probabilities. ProbLog facts, such as

$p::f.$ i fatti = affermazioni che sono vere

represent a statement f which is true with probability¹ p .
ProbLog rules, like $regole = affermazioni che sono vere se certe$
 $r \leftarrow c1, \dots, cn.$ condizioni sono vere

represent a property r inferred when $c1 \wedge \dots \wedge cn$ holds. Variable terms start with upper-case letters, constant terms with lower-case letters. Semicolon ; can be used to express OR conditions.

Each program defines a probability distribution over logic programs where a fact $p::f.$ is considered true with probability p and false with probability $1 - p$. The ProbLog engine [17] determines the success probability of a query q as the probability that q has a proof, given the distribution over logic programs. The engine also permits to automatically obtain a graphical representation of the AND-OR tree associated with the ground program that it used to answer a given query. This explains how the results are obtained. Finally, the ProbLog engine can be used in explanation mode to get, for each query, the list of all mutually exclusive proofs that lead to infer it.

3. Methodology and implementation

3.1. Overview

Fig. 1 offers an overview of the SecFog ingredients that will be thoroughly described in this section. First of all, SecFog considers two roles for its users:

- *infrastructure operators*, in charge of managing targeted Cloud-Edge nodes, and providing a description of the provisioned infrastructure capabilities to their users, and
- *application operators*, in charge of designing and managing application deployments by specifying their requirements (e.g., hardware, software, QoS), by monitoring deployment performance, and by re-distributing or re-configuring application services when needed.

Such Cloud-Edge stakeholders² might want to look for secure deployments of their applications and require explanations about such security assessment.

As per the SecFog input, infrastructure operators must provide a Node Descriptor (ND) for each of their managed nodes, declaring all the available security capabilities and their estimated effectiveness against attacks. These constitute the description of all Security Capabilities in the Cloud-Edge system.

On the other hand, application operators must provide a description of their Application and specify the Security Requirements of each application service (SR), in terms of the common vocabulary or, possibly, by means of a custom set of Security Policies declared over that vocabulary. The application operator can also possibly specify a complete or partial application deployment in case she wants to assess the security level of deployments under such constraints.

Finally, each stakeholder – infrastructure or application operator – can declare a trust degree towards any other stakeholder, as a set of opinions. Such weighted relations contribute to a Trust Network that is input to SecFog.

The SecFog prototype, given all Security Requirements, Security Capabilities and a Trust Network, features:

¹ A fact declared simply as $f.$ is assumed to be true with probability 1.

² Naturally, one stakeholder can play more than one role at a time and there can be more stakeholders playing the same role [19]. For instance, an infrastructure operator can provide IoT, Edge, or Cloud infrastructures to its customers, whilst managing applications and services over the very same infrastructure, or an application operator can act as an infrastructure provider when sharing her home router as an Edge capability for application deployment.

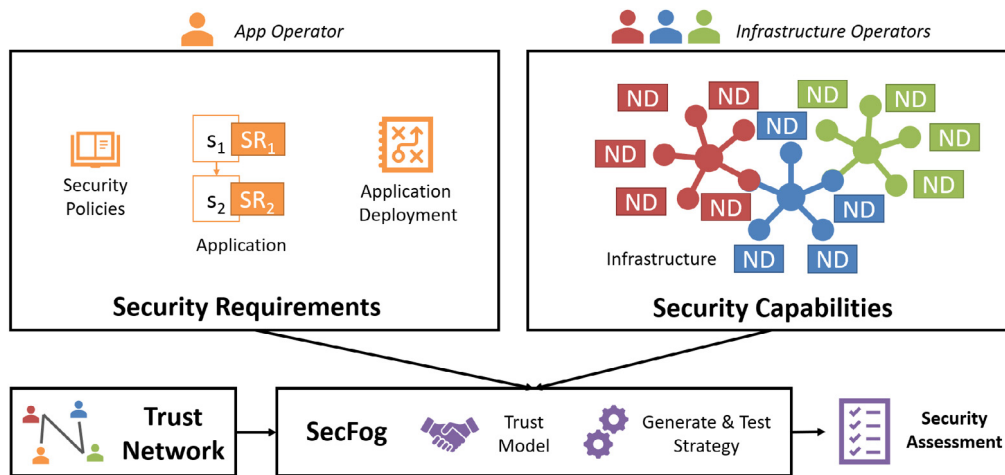


Fig. 1. Bird's-eye view of SecFog.

1. a **Generate & Test Strategy** to determine secure deployments by matching application requirements to infrastructure security capabilities (also considering their effectiveness against attacks), and
2. a **Trust Model** that is capable of completing the Trust Network, exploiting transitivity of trust chains and relying on the opinions declared by all stakeholders.

The output of SecFog is a **Security Assessment** of all eligible deployments determined through the exploration of the search space. For each candidate deployment, the prototype obtains its security level by multiplying the effectiveness of all exploited security capabilities, suitably weighted by trust degrees towards the operator providing them. Trust weights are crucial to mitigate and contrast the potential presence of operators that might declare unprecise, outdated or deliberately false data about the security capabilities they provide. As better detailed in Sections 3.4 and 5, our methodology allows to embed different trust models and to consider alternative interpretations for the values related to the effectiveness of available security capabilities. For instance, values declared by a certain provider can be overwritten by the results of objective measurements performed directly by application operators and based on their interactions with the provider, on asset locations, on the level of hardening of different host servers. Last but not least, each output comes with a proof that explains in a graphical, human-readable format how such result was obtained.

In the next paragraphs, we detail each of the ingredients we mentioned and, for each, we give the implementation³ as well as concrete, executable examples within our ProbLog prototype.

3.2. Security in Cloud-Edge computing

In Cloud-Edge computing infrastructures, end-to-end security must cover everything between the Cloud and the IoT. Naturally, the security requirements of multi-service applications will highly vary depending on business cases, target markets and vertical use cases as well as on the functionality offered by different components. For instance, a database storing video footage from domestic CCTV will probably require more security countermeasures to be active with respect to a database collecting outdoor temperature data from a city or neighbourhood. The OpenFog Consortium [20] highlighted the need for new Cloud-Edge computing platforms to guarantee privacy, anonymity, integrity, trust,

attestation, verification and measurement. Whilst security control frameworks exist for Cloud computing scenarios (e.g., the EU Cloud SLA Standardisation Guidelines [21] or the ISO/IEC 19086), to the best of our knowledge, no standard exists yet that defines security objectives for Cloud-Edge application deployments. Based on recent surveys about security aspects in the novel Cloud-Edge landscapes (i.e., [13,22,23]), we devised a simple example of taxonomy (Fig. 2) of security capabilities that can be offered by Cloud and Edge nodes and therefore used for reasoning on the security levels of given IoT application deployments.

Security capabilities that are common with the Cloud might assume renewed importance in Cloud-Edge scenarios, due to the limited resources of the devices installed closer to the edge of the Internet. For instance, guaranteeing physical integrity of and user data isolation at an access point with Edge capabilities might be very difficult. Dually, the possibility to encrypt or obfuscate data at Edge nodes, along with encrypted IoT communication and physical anti-tampering machinery, will be key to protect those application deployments that need data privacy assurance.

In the following description of the SecFog methodology, we will assume that all involved parties (viz., application and infrastructure operators) share the vocabulary of the example taxonomy⁴ in Fig. 2. Naturally, in an operational system based on SecFog, it will be necessary to adopt an extended and refined version of such taxonomy – with full definitions of the considered security capabilities – which we expect will be available as soon as normative security frameworks will get established for Cloud-Edge computing infrastructures.

3.2.1. Security capabilities: the infrastructure

The Infrastructure can be simply described by infrastructure operators as a set of facts declaring a node and its security capabilities, weighted by the probability that each capability can resist against malicious attacks. Such probability represents a measure of the effectiveness of the enforced security countermeasure. Fig. 3 lists the vocabulary of ProbLog facts that can be used to describe node capabilities in terms of the taxonomy of Fig. 2.

It is worth noting that undeclared capabilities are assumed to be unavailable at the considered node. Dually, some providers might decide not to disclose the effectiveness of security capabilities against malicious attacks. In such a case, they would be unfairly considered as the best ones by the system as plainly declared facts are assumed to be always true. To prevent this

³ The ProbLog code of SecFog prototype and of the examples of Sections 3 and 4 is publicly available at <https://github.com/di-unipi-socc/SecFog>.

⁴ Factually, different operators can employ different vocabularies and then exploit mediation [24] mechanisms, capable of translating one into another.

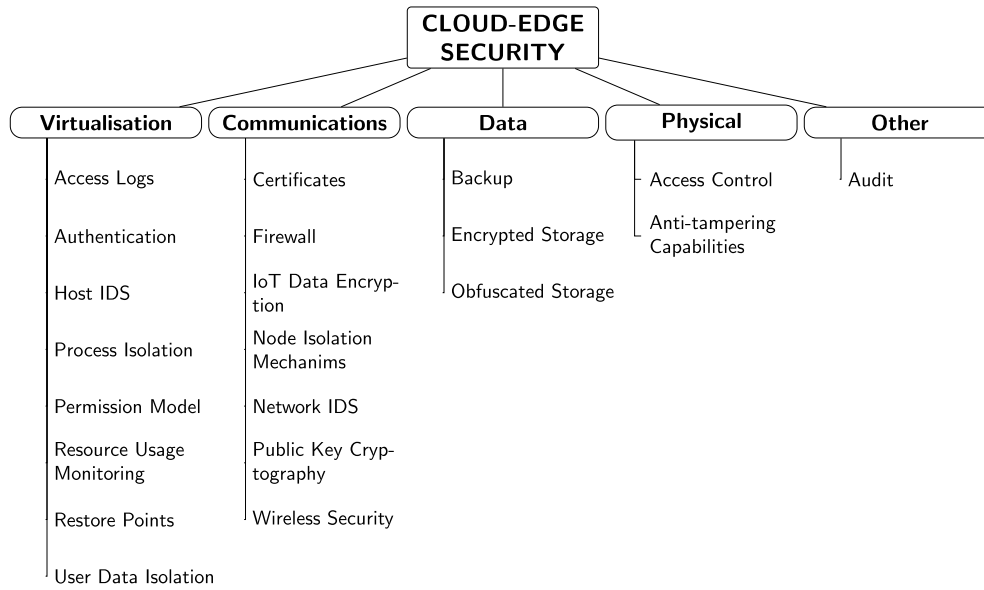


Fig. 2. An example of taxonomy of security capabilities in Cloud-Edge.

```

% virtualisation
access_logs(N).
authentication(N).
host_ids(N).
process_isolation(N).
permission_model(N).
resource_monitoring(N).
restore_points(N).
user_data_isolation(N).

% communication
certificates(N).
iot_data_encryption(N).
firewall(N).
node_isolation_mechanism(N).
network_ids(N).
public_key_cryptography(N).
wireless_security(N).

% data
backup(N).
encrypted_storage(N).
obfuscated_storage(N).

% physical
access_control(N).
anti_tampering(N).

% audit
audit(N).
  
```

Fig. 3. Example Cloud-Edge security capabilities in ProbLog.

from happening, application operators can for instance **rely on data collected from previous interactions** with the provider and specify probabilities on their own, or leverage the **adoption of a convenient trust model** (see Sections 3.4 and 5) **to mitigate the effects of such omissions**. Indeed, we reasonably expect that **providers declaring unreliable data will get lower trust** from application operators within the system.

Example. A Cloud node identified as `cloud1` and managed by a certain infrastructure operator `cloudOp1` can be specified in SecFog as:

```
node(cloud1, cloudOp1). "esiste un nodo chiamato cloud1, gestito da cloudOp1"
```

In case node `cloud1` features some form of firewall that is guaranteed to resist an attack (e.g., network traffic flood, packet fragmentation) with a likelihood of 99.99%, the previous line can be simply followed by:

```
0.9999::firewall(cloud1). "cloud1 ha un firewall efficace al 99.99%"
```

Similarly, an Edge node `edge3` managed by an operator `appOp42` and featuring a broken wireless security system like WEP and an encrypted storage considered 99% effective, is declared as:

```
node(edge3, appOp42).
0.01::wireless_security(edge3).
0.99::encrypted_storage(edge3).
```

Overall, this type of fact declarations can be used to specify the Node Descriptors by infrastructure operators, as sketched in Fig. 1. □

3.2.2. Security requirements: the application

As aforementioned, SecFog enables application operators to **specify an application along with the services that compose it**. Based on the same common vocabulary of Section 3.2, application operators can then **define (non-trivial) custom Security Policies that can be used to declare the Security Requirements of each service composing the application**. Custom security policies can

Riassunto: Per ogni servizio si dichiarano i requisiti usando il vocabolario della tassonomia. Si possono anche definire policy personalizzate, componendo capacità base.

be either existing ones, inferred from the presence of certain node capabilities, or they can be autonomously specified/enriched by the application operators, depending on business-related considerations.

Riassunto: Un'app viene descritta tramite i suoi servizi

Example. First, an application for smartfarming consisting of three services s1, s2 and s3 can be easily specified as:

```
app(smartfarming, [s1, s2, s3]).
```

Then, the application operator can decide that a node N offering backup capabilities together with encrypted_storage or obfuscated_storage can be considered a secureStorage provider. The custom security policy described above can be specified as:

```
secureStorage(N) :- "un nodo N offre secureStorage se ha backup E (storage cifrato O storage offuscato)"
    backup(N), AND
    (encrypted_storage(N); obfuscated_storage(N)).
```

A different stakeholder might also require the availability of a certificate at the node featuring secure storage and re-define the policy as:

```
secureStorage(N) :-
    backup(N),
    certificate(N),
    (encrypted_storage(N); obfuscated_storage(N)).
```

The Security Requirements for service s2 of smartfarming, which needs both secureStorage (as previously declared) and resource_monitoring capabilities can then be defined as:

```
securityRequirements(s2, N) :- Policy per definire i requisiti
    secureStorage(N),           Il servizio s2 richiede che il nodo N
    resource_monitoring(N).      offra secureStorage E
                                resource_monitoring.
```

Overall, this permits application operators to quickly and flexibly specify all the Security Requirements of their software systems by exploiting combinations of custom Security Policies and basic security capabilities. □

3.3. Generate & Test strategy

The ProbLog listing in Fig. 4 defines the declarative Generate & Test strategy of SecFog. This strategy exploits a knowledge base of facts and rules that defines a Cloud-Edge infrastructure and its Security Capabilities along with an application and its Security Requirements, declared as we have seen in the previous section. It is worth noting that SecFog strategy can be used both

- (a) to determine (or complete) secure application deployments, assessing their security level, and
- (b) to assess the security level guaranteed by complete input deployments, which may be already used by the application operators.

In both cases (a) and (b), the quantitative security assessment considers the available security capabilities and their declared effectiveness against attacks.

The rule for secFog(OpA, A, D) inputs an application operator OpA, an application A she is in charge of deploying and a (possibly empty, or partial) deployment D of such application. First, it checks that A has been declared as an application app(A, L) (line 2), then it evaluates the predicate deployment(OpA, L, D) (line 3). Recursively, for each component C in the list of the application components, it checks whether it (can be) has been deployed to a node N (line 7) that can satisfy the security requirements declared by the application operator for C, i.e. whether securityRequirements(C, N) holds (line 8).

```
secFog(OpA, A, D) :- (1)
    app(A, L), (2)
    deployment(OpA, L, D). (3)
    (4)
    deployment(_, [], []). (5)
    deployment(OpA, [C|Cs], [d(C, N, OpN) | D]) :- (6)
        node(N, OpN), (7)
        securityRequirements(C, N), (8)
        deployment(OpA, Cs, D). (9)
```

Fig. 4. The Generate & Test strategy of SecFog.

genera automaticamente tutte le combinazioni possibili di assegnamenti nodo-servizio, e per ognuna verifica se i requisiti sono soddisfatti.

Example. Considering a single-service application, managing the weather data of a municipality, and an infrastructure composed of two (one Cloud and one Edge) nodes declared as follows:

```
%%% Application, specified by appOp
app(weatherApp, [weatherMonitor]).
securityRequirements(weatherMonitor, N) :-
    (anti_tampering(N); access_control(N)),
    (wireless_security(N); iot_data_encryption(N)).
```

```
%%% Cloud node, specified by cloudOp
node(cloud, cloudOp).
0.99::anti_tampering(cloud).
0.99::access_control(cloud).
0.99::iot_data_encryption(cloud).
```

```
%%% Edge node, specified by edgeOp
node(edge, edgeOp).
0.8::anti_tampering(edge).
0.9::wireless_security(edge).
0.9::iot_data_encryption(edge).
```

Running the query

```
query(secFog(appOp, weatherApp, D)).
```

outputs the resulting secure deployments for the weather-App, along with a value in the range [0, 1] that represents their assessed security level (based on the declared effectiveness of infrastructure capabilities that are exploited by each possible deployment):

```
secFog(appOp, weatherApp, [d(weatherMonitor, cloud, cloudOp)]) : 0.989901
secFog(appOp, weatherApp, [d(weatherMonitor, edge, edgeOp)]) : 0.792
```

The result, highlighting the deployment to the Cloud as the most secure solution, can be explained by looking at Fig. 5, which graphically depicts the AND-OR trees of the two ground programs that lead to the output results. Such graphical explanations can be obtained automatically, by using ProbLog in ground mode.⁵ Note that the ProbLog engine performs an AND-OR graph search over the ground program to determine the query results. For instance, the value associated with securityRequirements(weatherMonitor, cloud) is obtained as:

$$\begin{aligned}
 & p(\text{anti_tampering}(\text{cloud})) \\
 & \times p(\text{iot_data_encryption}(\text{cloud})) + \\
 & (1 - p(\text{anti_tampering}(\text{cloud}))) \\
 & \times p(\text{access_control}(\text{cloud})) \\
 & \times p(\text{iot_data_encryption}(\text{cloud})) = \\
 & .99 \times .99 + (1 - .99) \times .99 \times .99 = \\
 & 0.989901
 \end{aligned} \tag{1}$$

As for the AND-OR graph of the ground program, also this proof can be obtained automatically, by using ProbLog in explain mode.⁶ □

⁵ <https://problog.readthedocs.io/en/latest/cli.html#grounding-ground>

⁶ <https://problog.readthedocs.io/en/latest/cli.html#explanation-mode-explain>

// ricorsiva: prende il primo servizio della lista [C|Cs], cerca un nodo N che soddisfi i suoi requisiti di sicurezza, lo assegna, e poi ripete per il resto della lista.

Nota: da riguardare i grafici

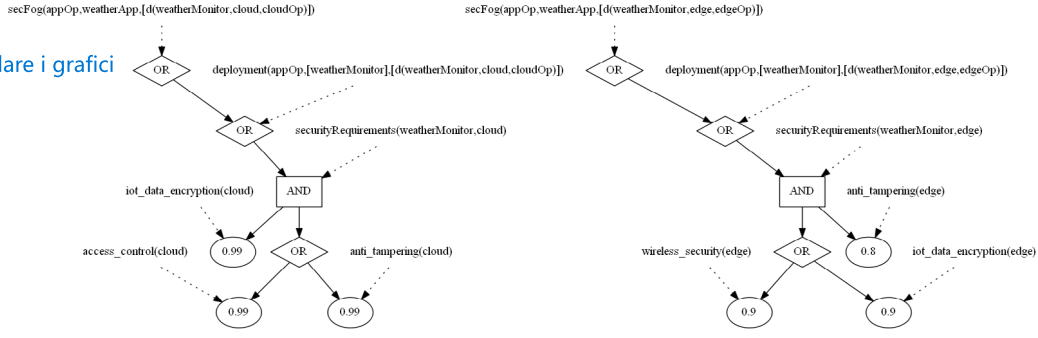


Fig. 5. Graphical ground program of the weatherApp example.

```

trusts(X,X).          (1)
                      (2)
trusts2(A,B) :-      (3)
    trusts(A,B).      (4)
trusts2(A,B) :-      (5)
    trusts(A,C),      (6)
    trusts2(C,B).     (7)

```

Fig. 6. Default trust model of SecFog.

```

deployment(_,[],[]). (1)
deployment(OpA,[C|Cs],[d(C,N,OpN)|D]) :- (2)
    node(N,OpN), (3)
    securityRequirements(C,N), (4)
    trusts2(OpA,OpN), (5)
    deployment(OpA,Cs,D). (6)

```

Fig. 7. The deployment/3 predicate with trust.

3.4. Default trust model

The pervasive and highly distributed nature of new Cloud-Edge deployments imposes to deal not only with the effectiveness of the adopted security capabilities but also with the **trust degrees towards various, potentially unknown, infrastructure operators**. As aforementioned in Section 3, **trust considerations are also essential both to discredit unreliable (e.g., lazy or dishonest) providers that declare unprecise, outdated or false data about their assets, and to discourage others to do so**. In this section, we show how it is possible to smoothly extend the prototype SecFog described up to now so to include a simple, yet powerful, probabilistic trust model. In Section 5, we will detail how SecFog can be generalised to accommodate and leverage more complex trust models.

The default trust model of SecFog considers **direct trust relations between two stakeholders A and B as the probability that A can trust B, e.g. based on an aggregate of all previous interactions they had**. The proposed trust model combines such direct trust opinions from different stakeholders and completes the (possibly partial) trust network input to SecFog with missing indirect trust relations. Specifically, the default trust model considers **trust relations as transitive** and explores network paths **while aggregating the opinions** (as declared by application and infrastructure operators). **Opinions along paths are (unconditionally) combined via multiplication, opinions across paths are (monotonically) combined via addition**. Intuitively, this causes **opinions to deteriorate along paths** and, when multiple opinions are available, to improve, by weighting more those paths that are more trustworthy [25].

Fig. 6 lists the Prolog rules we used to define our default model which is a (probabilistic) transitive closure of the trust network input to SecFog. Trivially, **we assume that each stakeholder fully trust herself** (line 1). Then, **stakeholder A can trust B either directly (lines 3–4), or through a third party C that directly or indirectly trusts B (lines 5–7)**.

Fig. 7 shows how to include the trust model in the Generate and Test strategy of Fig. 4, namely by **simply adding the condition trusts2(OpA, OpN)**.

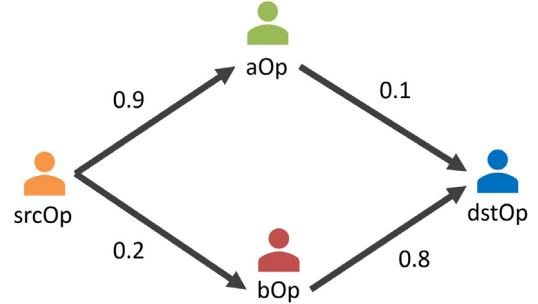


Fig. 8. A trust network.

We first show an example of this trust model alone, then we apply it to the example of Section 3.3 to illustrate its usage within SecFog.

Example. Consider the trust network of Fig. 8 and suppose to be interested in the (indirect) trust relationship between srcOp and dstOp.

It can be simply computed with the SecFog trust model in Prolog as:

```

%%% trust relations declared by srcOp
0.9::trusts(srcOp, aOp).
0.2::trusts(srcOp, bOp).

%%% trust relations declared by aOp
0.1::trusts(aOp, dstOp).

%%% trust relations declared by bOp
0.8::trusts(bOp, dstOp).

query(trusts2(srcOp, dstOp)).

```

which returns

```
trusts2(srcOp,dstOp): 0.2356
```

as a result. Also in the case of the trust model, it is possible to get the ground program which explains how the final result was computed (Fig. 9).

From this simple example it is clear that, in the default trust model, the contribution of trust relations **deteriorates along paths** and that **all possible paths give their contribution to the output**.

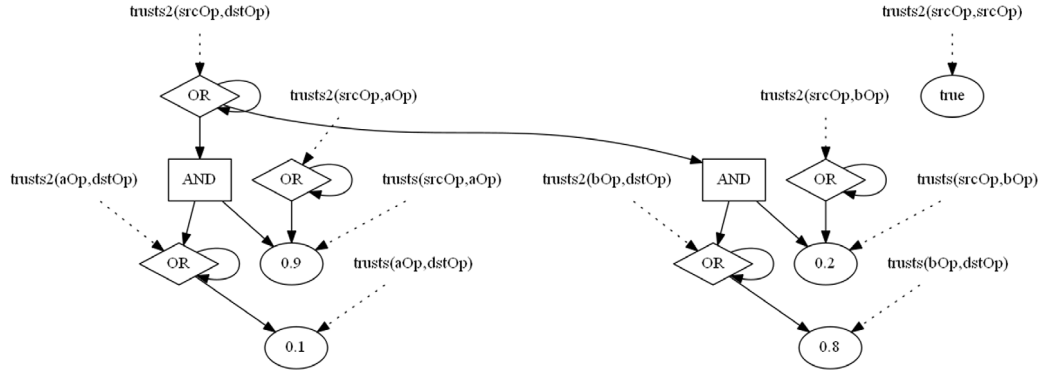


Fig. 9. Graphical ground program of the trusts2 example.

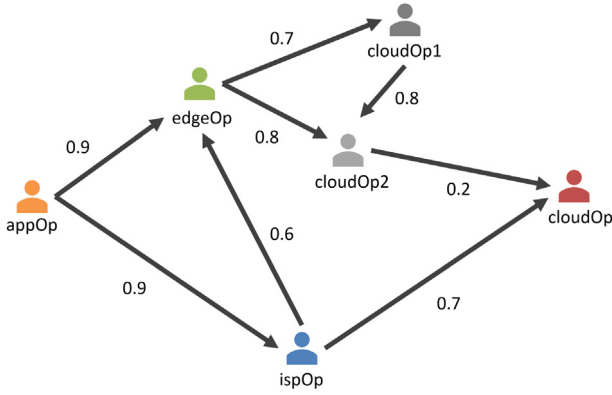


Fig. 10. Example trust network among Cloud-Edge operators.

result. Indeed, the final result corresponds to the likelihood that it is possible to establish a *trust path* from *srcOp* to *dstOp* over the considered trust network. □

Example. We now retake the example of Section 3.3 and we solve it again by also taking into account the trust network of Fig. 10.

The network is defined by the direct trust relations, which are declared by the different operators as:

```
%% trust relations declared by appOp
.9::trusts(appOp, edgeOp).
.9::trusts(appOp, ispOp).

%% trust relations declared by edgeOp
.7::trusts(edgeOp, cloudOp1).
.8::trusts(edgeOp, cloudOp2).

%% trust relation declared by cloudOp1
.8::trusts(cloudOp1, cloudOp2).

%% trust relation declared by cloudOp2
.2::trusts(cloudOp2, cloudOp).

%% trust relations declared by ispOp
.8::trusts(ispOp, cloudOp).
.6::trusts(ispOp, edgeOp).
```

The same query of the previous example now leads to a different result:

```
secFog(appOp, weatherApp, [d(weatherMonitor, cloud, cloudOp)]):
0.76017935
secFog(appOp, weatherApp, [d(weatherMonitor, edge, edgeOp)]):
0.755568
```

When accounting for trust, output security levels are lower than those obtained by considering only the effectiveness of security capabilities (0.76 and 0.75 vs. 0.98 and 0.79), and the Cloud

deployment does not outperform the Edge deployment anymore. In situations like this one, the application operator might make her choice also considering other estimated non-functional parameters (e.g., cost, response time, resource usage). □

Before concluding this section, it is worth noting that probabilities related to the effectiveness of security countermeasures as well as those defining trust relations can also be derived from more complex models and input to SecFog. Indeed, the effectiveness of security countermeasures can either be extracted from the SLAs of each infrastructure provider or, alternatively, be obtained from objective measurements and data collected by application operators from previous interactions with infrastructure providers. On the other hand, trust degrees – which represent a subjective piece of information – can be either defined by the application operators or derived from more complex trust models.

On this line, for instance, SecFog can naturally embed a simplified formulation of the trust evaluation method by Tang et al. [26]. Namely, ProbLog facts representing the security capabilities featured by a node, like

```
0.9999::firewall(cloud1).
```

can be exploited to represent objective trust assessments of services (computed in [26] as average conformance values between monitored and claimed QoS of a service). ProbLog facts representing the trust relation between stakeholders, like

```
0.9::trusts(appOp, edgeOp).
```

can be exploited to represent subjective trust assessments of service operators (actually computed in [26] as measures of the trust of a user on a service). A simple combination of objective and subjective trust assessment will be then directly performed by ProbLog while evaluating the deployment/3 predicate.⁷

Other simple extensions to the described probabilistic trust model consist of conditioning trust transitivity to the absence of direct trust relations, or of allowing trust transitivity in the trust network only within a specified distance (i.e. radius) from the application operator (as epitomised in Section 5).

In the next section, we exploit SecFog to analyse a lifelike example of IoT application deployment to Cloud-Edge infrastructure. We also make use of our FogTorchΠ prototype [27] to select deployments that also meet hardware and software requirements of the example application.

⁷ Please note that the purpose of the previous discussion is merely to illustrate how a simplified formulation of (the results of) the trust evaluation method of [26] can be embedded into our methodology. In fact, the trust evaluation method of [26] is more sophisticated than what we described, as it computes subjective trust assessments by taking into account both the feedback ratings of an user and those from similar, trustworthy users, and as it combines objective and subjective trust assessments by considering also their confidence.

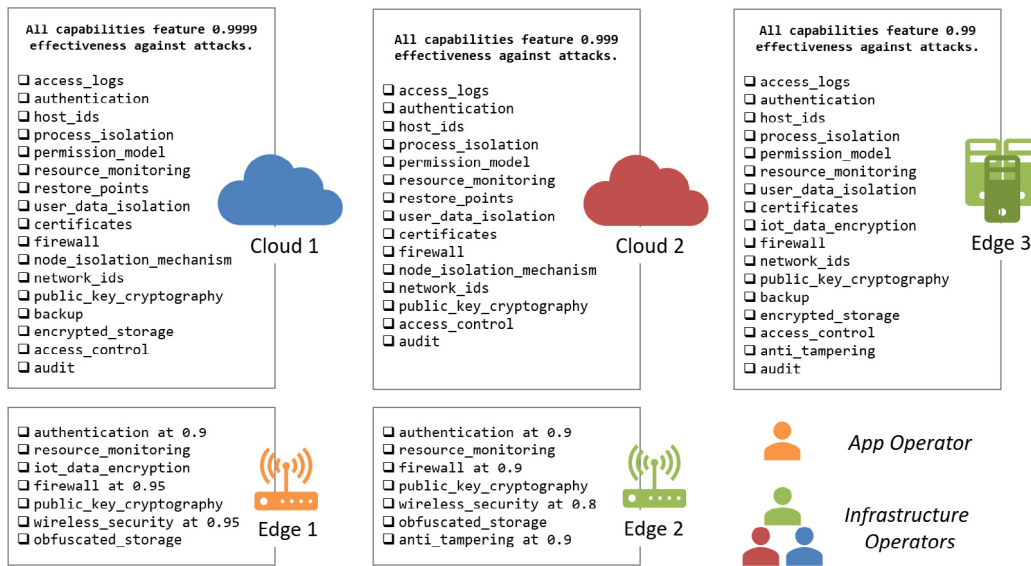


Fig. 11. Cloud-Edge infrastructure security capabilities.

4. Motivating example

4.1. Infrastructure

Fig. 11 shows the Cloud-Edge infrastructure – two Cloud data centres, three Edge nodes – to which a smart building application is to be deployed. For each node, the available security capabilities and their effectiveness against attacks⁸ (as declared by the infrastructure operator) are listed in terms of the taxonomy of Fig. 3.

Relying on such information, node descriptors can be easily expressed by each infrastructure operator through listing ground facts, as discussed in Section 3.2.1. For instance, edge1 directly operated by the application operator appOp is described as

```
node(edge1, appOp).
0.9::authentication(edge1).
resource_monitoring(edge1).
iot_data_encryption(edge1).
0.95::firewall(edge1).
public_key_cryptography(edge1).
0.95::wireless_security(edge1).
obfuscated_storage(edge1).
```

i nodi Cloud hanno praticamente tutto, i nodi Edge hanno un sottoinsieme delle capacità con efficacia inferiore.

All the Node Descriptors assembled following this template form the description of the security capabilities available in the infrastructure.

4.2. Application

We retake the application example of [27] and we extend it with security requirements. Consider a simple multi-service IoT application (Fig. 12) that manages fire alarm, heating and A/C systems, interior lighting, and security cameras of a smart building. The application consists of three microservices:

- **IoTController**, interacting with the connected cyber-physical systems,
- **DataStorage**, storing all sensed information for future use and employing machine learning techniques to update sense-act rules at the IoTController so to optimise heating and lighting management based on previous experience and/or on people behaviour, and

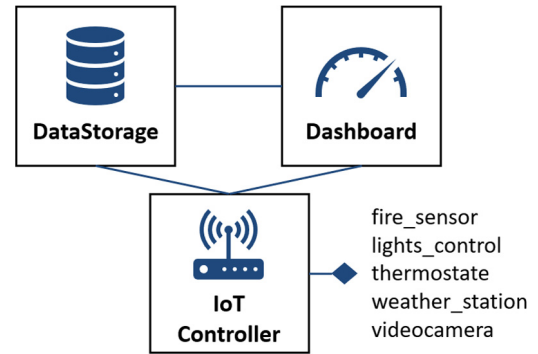


Fig. 12. Multi-service IoT application.

- **Dashboard**, aggregating and visualising collected data and videos, as well as allowing users to interact with the system.

Each microservice represents an independently deployable component of the application [28] and has its own security requirements.

Particularly, application operators defined the following security requirements:

- **IoTController** requires physical_security guarantees (i.e., access_control $\vee$ anti_tampering) so to avoid that temporarily stored data can be physically stolen from the deployment node,
- **DataStorage** requires secure_storage (viz., backup $\wedge$ (obfuscated_storage $\vee$ encrypted_storage)), the availability of access_logs, a network_ids in place to prevent distributed Denial of Service (DDoS) attacks, and
- **Dashboard** requires a host_ids installed at the deployment node (e.g., an antivirus software) along with a resource_monitoring to prevent interactions with malicious software and to detect anomalous component behaviour.

Furthermore, the application requires guaranteed end-to-end encryption among all services (viz., all deployment nodes should feature public_key_cryptography) and that deployment nodes should feature an authentication mechanism.

⁸ In Fig. 11 when the effectiveness against attacks of a capability is not indicated we assume it is considered to be 1 by the corresponding infrastructure provider.

The described application and security policies translate one-to-one to the following SecFog clauses, as discussed in Section 3.2.2:

```
%%% application
app(smartbuilding, [iot_controller, data_storage, dashboard]).

%%% security requirements
securityRequirements(iot_controller, N) :-
    physical_security(N),
    public_key_cryptography(N),
    authentication(N).

securityRequirements(data_storage, N) :-
    secure_storage(N),
    access_logs(N),
    network_ids(N),
    public_key_cryptography(N),
    authentication(N).

securityRequirements(dashboard, N) :-
    host_ids(N),
    resource_monitoring(N),
    public_key_cryptography(N),
    authentication(N).

%%% custom policies
physical_security(N) :-
    anti_tampering(N); access_control(N).

secure_storage(N) :-
    backup(N),
    (encrypted_storage(N); obfuscated_storage(N)).
```

4.3. Trust network

Finally, as discussed in Section 3.4, we consider the trust network of Fig. 13, which can be defined as:

```
%%% trust relations declared by appOp
0.9::trusts(appOp, edgeOp).
0.8::trusts(appOp, cloudOp2).

%%% trust relations declared by edgeOp
0.9::trusts(edgeOp, cloudOp2).
0.7::trusts(edgeOp, cloudOp1).

%%% trust relations declared by cloudOp2
0.1::trusts(cloudOp1, cloudOp2).

%%% trust relations declared by cloudOp2
0.8::trusts(cloudOp2, edgeOp).
0.5::trusts(cloudOp2, cloudOp1).
```

Accounting for trust propagation, such network results in the following value of trust of appOp towards infrastructure providers:

```
trusts2(appOp, appOp):      1
trusts2(appOp, cloudOp1):   0.8247
trusts2(appOp, cloudOp2):   0.96326
trusts2(appOp, edgeOp):     0.964
```

4.4. Security assessment

As discussed in Section 3.3, the SecFog prototype can be used to find all deployments that satisfy the security requirements of the example application to the given infrastructure, by simply issuing the query:

```
query(secFog(appOp, smartbuilding, D)).
```

As shown in Table 1, relying on ProbLog out-of-the-box algorithms, SecFog prototype returns answers to the query along with a value in [0, 1] that represents the aggregate security level of the inferred facts, i.e. the probability that a deployment can be

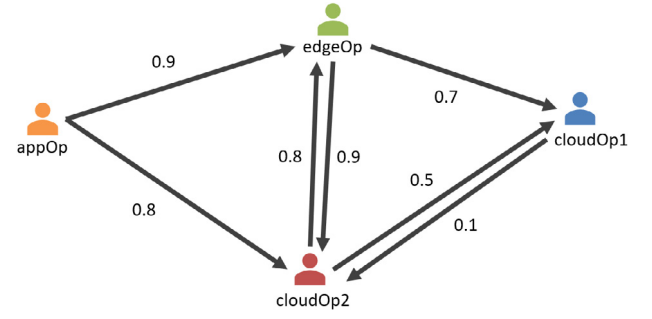


Fig. 13. Trust network of the smartbuilding example.

Table 1

Eligible deployments of the example application.

Dep. ID	IoTController	DataStorage	Dashboard	Security
Δ1	Cloud 1	Cloud 1	Cloud 1	0.82
Δ2	Cloud 1	Cloud 1	Cloud 2	0.81
Δ3	Cloud 1	Cloud 1	Edge 3	0.78
Δ4	Cloud 1	Edge 3	Cloud 1	0.77
Δ5	Cloud 1	Edge 3	Cloud 2	0.75
Δ6	Cloud 1	Edge 3	Edge 3	0.75
Δ7	Cloud 2	Cloud 1	Cloud 1	0.81
Δ8	Cloud 2	Cloud 1	Cloud 2	0.81
Δ9	Cloud 2	Cloud 1	Edge 3	0.77
Δ10	Cloud 2	Edge 3	Cloud 1	0.75
Δ11	Cloud 2	Edge 3	Cloud 2	0.89
Δ12	Cloud 2	Edge 3	Edge 3	0.87
Δ13	Edge 2	Cloud 1	Cloud 1	0.66
Δ14	Edge 2	Cloud 1	Cloud 2	0.65
Δ15	Edge 2	Cloud 1	Edge 3	0.63
Δ16	Edge 2	Edge 3	Cloud 1	0.62
Δ17	Edge 2	Edge 3	Cloud 2	0.72
Δ18	Edge 2	Edge 3	Edge 3	0.72
Δ19	Edge 3	Cloud 1	Cloud 1	0.80
Δ20	Edge 3	Cloud 1	Cloud 2	0.78
Δ21	Edge 3	Cloud 1	Edge 3	0.78
Δ22	Edge 3	Edge 3	Cloud 1	0.77
Δ23	Edge 3	Edge 3	Cloud 2	0.89
Δ24	Edge 3	Edge 3	Edge 3	0.89

considered secure according to the declared reliability of the infrastructure capabilities and to the trust degree of the application operator towards each exploited infrastructure operator.

If the application operator is only considering security as a parameter to lead her search, she would try to maximise the obtained metric and, most probably, select one among Δ11, Δ12, Δ23, Δ24. However, security might need to be considered along with other parameters so to find a suitable trade-off among them.

In this regards, it is interesting to see how SecFog prototype can be used in synergy with other tools that perform multi-service application placement in Cloud-Edge scenarios. For instance, our FogTorchΠ prototype [3] finds eligible deployments that guarantee software, hardware and network QoS requirements. For each deployment, it outputs the QoS-assurance (i.e., the likelihood it will meet network QoS requirements), an aggregate measure of Edge resource consumption, and an estimate of its monthly operational cost. It then employs a simple multi-objective optimisation to rank the deployments and to decide which are the better candidates.

For the smartbuilding example we are analysing, among the deployments of Table 1, FogTorchΠ suggests only Δ13, Δ16 and Δ22 [27]. Naturally, with the aim of maximising security whilst considering all other requirements, the application operator would likely choose Δ22.

5. Algebraic extensions of SecFog

The default probabilistic trust model of SecFog shows two main limitations. Namely, it is **unconditionally transitive** (i.e., if A trusts B and B trusts C then A trusts C) and **monotonic** (all paths towards a certain provider P contribute increasing the trust degree towards it).

More often, trust has been considered only **conditionally transferable** and – being a **subjective** phenomenon – usually **non-monotonically increasing** [29]. As we will show in this section, the default trust model of SecFog can be easily generalised so to accommodate semiring-based trust models such as those proposed by Theodorakopoulos and Baras [25], Bistarelli et al. [30,31] or Gao et al. [32], in fields other than multi-service application deployment.

After briefly reviewing the mathematical definition of semirings and describing the algebraic extension of ProbLog by Kimmig et al. [33], we show how SecFog can be generalised into an algebraic α SecFog and embed different semiring-based trust models. We then exploit two among those models to solve again the motivating example of Section 4.

A (commutative) **semiring** is an algebraic structure consisting of a 5-tuple:

$$\langle S, \oplus, \otimes, \mathbf{0}, \mathbf{1} \rangle$$

where S is a set of elements, and $\oplus$ and $\otimes$ are two binary operators defined over such elements such that:

- $\oplus$ is commutative and associative, and $\mathbf{0}$ is its neutral element,
- $\otimes$ is associative, distributes over $\oplus$, and $\mathbf{1}$ and $\mathbf{0}$ are its neutral and absorbing elements, respectively.

For instance, the embedded model of ProbLog corresponds to the probability semiring

$$\langle \mathbb{R} \cap [0, 1], +, \times, 0, 1 \rangle$$

where $+$ and $\times$ denote classical addition and multiplication over real numbers.

Intuitively, a ProbLog program leverages input probability distributions to analyse all possible Prolog programs (i.e., worlds) that could be generated according to them. Assuming that $\Omega(q)$ is the set of possible worlds W that entail a valid proof for a certain query q (i.e., $\Omega(q) = \{W \mid W \models q\}$), the ProbLog engine computes the probability $p(q)$ that q holds as

$$p(q) = \sum_{W \in \Omega(q)} \prod_{f \in W} p(f)$$

where f are facts within a certain possible world, and $p(f)$ is the probability they are labelled with. More generally, the algebraic extension of SecFog allows programmers to **rely on arbitrary semirings and labelling functions** for computing output results. Intuitively, given an arbitrary semiring $\langle S, \oplus, \otimes, \mathbf{0}, \mathbf{1} \rangle$ as defined above and a labelling function $\alpha(f)$ over the program literals, then the labelling for query q is obtained as

$$A(q) = \bigoplus_{W \in \Omega(q)} \bigotimes_{f \in W} \alpha(f)$$

where worlds in Ω represent the set of **interpretations** where q is true. For all details on the algebraic extension of ProbLog we refer the reader to [33].

In what follows, we exploit such feature of ProbLog to rely on different trust models and solve the motivating example of Section 4.

Example. We start by embedding in α SecFog the trust model proposed in [25] and [30]. Still relying on the transitive closure of the trust network, their model exploits a semiring where trust is represented by couples $\langle t, c \rangle$ in the set $S = (\mathbb{R} \cap [0, 1]) \times (\mathbb{R} \cap [0, 1])$ where t represents a trust value and c the confidence in such trust value assignment, i.e. the **quality of the declared opinion**. Then, $\otimes$ (with neutral element $\langle 1, 1 \rangle$) is defined as

$$\langle t, c \rangle \otimes \langle t', c' \rangle = \langle t \times t', c \times c' \rangle$$

and $\oplus$ (with neutral element $\langle 0, 0 \rangle$) is defined as

$$\langle t, c \rangle \oplus \langle t', c' \rangle = \begin{cases} \langle t, c \rangle & \text{if } c > c' \\ \langle t', c' \rangle & \text{if } c' > c \\ \langle \max\{t, t'\}, c \rangle & \text{if } c = c' \end{cases}$$

This model overcomes the limitations of the default trust model as it conditions trust transitivity to the confidence values and avoids monotonicity by bounding the trust value to the maximum declared one.

When embedding this trust model in α SecFog, we obtain that effectiveness of security countermeasures can be declared by infrastructure providers as in

```
(0.9999,1)::firewall(cloud1).
```

and that trust relations take the form

```
(0.9,0.8)::trusts(appOp, edgeOp).
```

We run α SecFog with this trust model over the motivating example of Section 4, assuming that declared trust opinions are associated with confidence values as follows:

```
%% trust relations declared by appOp
(0.9,0.9)::trusts(appOp, edgeOp).
(0.8,0.9)::trusts(appOp, cloudOp2).
```

```
%% trust relations declared by edgeOp
(0.9,0.9)::trusts(edgeOp, cloudOp2).
(0.7,0.5)::trusts(edgeOp, cloudOp1).
```

```
%% trust relations declared by cloudOp2
(0.1,0.9)::trusts(cloudOp1, cloudOp2).
```

```
%% trust relations declared by cloudOp2
(0.8,0.7)::trusts(cloudOp2, edgeOp).
(0.5,0.7)::trusts(cloudOp2, cloudOp1).
```

Querying deployment/3 results in Δ_{13} having an overall security level of $\langle 0.116, 0.357 \rangle$, Δ_{16} having an overall security level of $\langle 0.247, 0.510 \rangle$ and Δ_{22} having an overall security level of $\langle 0.296, 0.510 \rangle$.

Δ_{22} is the most likely deployment to be chosen by the application operators as it is ranked first also with the introduction of the new model. Akin results are obtained when substituting the max with the min operator in the semiring, which makes the computation of the security level less optimistic when confidence values coincide. $\square$

Example. Building on the previous model, Gao et al. [32] recently proposed a more sophisticated one, capable of considering both trust (i.e., positive preference) and distrust (i.e., negative preference) relations. Their model also relies on a semiring where (dis)trust is represented by couples $\langle t, c \rangle$ in the set $\mathcal{T} = (\mathbb{R} \cap [-1, 1]) \times (\mathbb{R} \cap [0, 1])$, where t represents a (dis)trust value and c the confidence in such value assignment. Distrust ranges in $[-1, 0]$ and trust in $(0, 1]$, having 0 represent an *indifferent* opinion. Then, $\otimes$ (with neutral element $\langle 1, 1 \rangle$) is defined as

$$\langle t, c \rangle \otimes \langle t', c' \rangle = \begin{cases} \langle 0, c \times c' \rangle & \text{if } t < 0 \text{ and } t' < 0 \\ \langle t \times t', c \times c' \rangle & \text{otherwise} \end{cases}$$

```

trusts2(A,B) :- trusts2(A,B,3).      (1)
trusts2(A,B,D) :-                    (2)
    D > 0,                            (3)
    trusts(A,B).                     (4)
trusts2(A,B,D) :-                    (5)
    D > 0,                            (6)
    trusts(A,C),                     (7)
    NewD is D - 1,                   (8)
    trusts2(C,B,NewD).               (9)

```

Fig. 14. The trust2/3 predicate with maximum propagation radius D= 3.

and $\oplus$ (with neutral element $\langle 0, 0 \rangle$) is defined as

$$\langle t, c \rangle \oplus \langle t', c' \rangle = \begin{cases} \langle t, c \rangle & \text{if } c > c' \\ \langle t', c' \rangle & \text{if } c' > c \\ \langle \text{sign}(t + t') \cdot \max\{t, t'\}, c \rangle & \text{if } c = c' \end{cases}$$

where $\text{sign}(x)$ returns 1 if $x \geq 0$ and -1 otherwise.

This trust model therefore permits to express both trust and distrust opinions. It is worth mentioning that the $\otimes$ operator sets trust to 0 when both considered opinions represent distrust along a path, what stops trust transitivity. Also, the authors of [32] propose to impose a maximum radius D for propagating (dis)trust relations, which can be easily obtained in SecFog by revising the trust2 predicate as listed in Fig. 14.

We now run the motivating example with the new trust model and D set to 3, assuming that Cloud providers decide to declare the following distrust opinions towards each other (instead of the trust opinions previously specified):

```

(-0.1,0.9)::trusts(cloud0p1, cloud0p2).
(-0.1,0.7)::trusts(cloud0p2, cloud0p1).

```

As a result, the three eligible deployments Δ_{13} , Δ_{16} and Δ_{22} obtain security levels of $\langle 0.005, 0.357 \rangle$, $\langle 0.05, 0.510 \rangle$ and $\langle 0.06, 0.510 \rangle$ respectively. Despite Δ_{22} ranks first, such results, very close to 0 (i.e. to an indifferent opinion), might induce the application operators to consider upgrading part of the infrastructure they manage so to permit deployment to their assets, or to include other providers in their analysis. $\square$

6. Related work

Among the studies focussing on the placement of multi-service applications to Cloud nodes, very few approaches considered security aspects when determining eligible application deployments, mainly focussing on improving performance, resource usage and deployment cost [22,34], or on identifying potential data integrity violations based on pre-defined risk patterns [35]. Other existing research considered security mainly when treating the deployment of business processes to (federated) multi-Clouds (e.g., [36–38]). Similar to our work, Luna et al. [39] were among the first to propose a quantitative reasoning methodology to rank single Cloud providers based on their security SLAs, and with respect to a specific set of (user-weighted) security requirements. Recently, swarm intelligence techniques [22] have been exploited to determine eligible deployments of composite Cloud applications, considering a risk assessment score based on node vulnerabilities. However, none of these works embedded trust models to consider the trust relations and the opinions of the involved stakeholders when determining secure deployments.

Cloud-Edge computing introduces new challenges, mainly due to its pervasive geo-distribution and heterogeneity, need for QoS-awareness, dynamicity and support to interactions with the IoT, that were not thoroughly studied in previous works addressing

the problem of application deployment to the Cloud [40,41]. Among the first proposals investigating these new lines, [5] proposed a Cloud-Edge search algorithm as a first way to determine an eligible deployment of (multi-component) DAG applications to tree-like infrastructures. Their placement algorithm attempts the placement of services Edge-to-Cloud by considering hardware capacity only. An open-source simulator – iFogSim – has been released to test the proposed policy against Cloud-only deployments. Building on top of iFogSim, [42] tried to guarantee the application service delivery deadlines and to optimise computational resource exploitation. Also [43] used iFogSim to implement an algorithm for optimal online placement of application components, with respect to load balancing. Recently, exploiting iFogSim, [6] proposed a distributed search strategy to find the best service placement in Cloud-Edge infrastructures, which minimises the distance between the clients and the most requested services, based on request rates and available free resources. In our previous work, we also proposed a model and algorithms to determine eligible deployments of IoT applications to Fog infrastructures [8] based on hardware, software and network QoS requirements. Our prototype – FogTorchPI – implements those algorithms and permits to estimate the QoS-assurance, the resource consumption in the Fog layer [9] and the monthly deployment cost [44] of the output eligible deployments. [45,46] proposed (linearithmic) heuristic algorithms that attempt deployments prioritising placement of applications to devices that feature with less free resources.

From an alternative viewpoint, [47] gave a Mixed-Integer Non-Linear Programming (MINLP) formulation of the problem of placing application services aiming at satisfying end-to-end delay constraints. The problem is then solved by linearisation into a Mixed-Integer Linear Programming (MILP), showing potential improvements in latency, energy consumption and costs for routing and storage that the Cloud-Edge interplay might bring. Also [7] adopted an ILP formulation of the problem of allocating computation to Cloud and Edge nodes so to optimise time deadlines on application execution. A simple linear model for Cloud costs is also taken into account. Similar solutions were proposed, attempting to optimise various metrics such as access latency, resource usage, energy consumption or data migrations cost [48–53]. [54] described instead a fuzzy QoE extension of iFogSim – based on an ILP modelling of users expectation – which achieved improvements in network conditions and service QoS. Regrettably, none of the discussed ILP/MILP approaches came with the code to run the experiments. Conversely, [55] proposed a software platform to support optimal application placement in Cloud-Edge landscapes. Envisioning resource, bandwidth and response time constraints, they compare a Cloud-only, an Edge-only or a Cloud-to-Edge deployment policy. Additionally, the authors of [56] released an open-source extension of Apache Storm that performs service placement while improving the end-to-end application latency and the availability of deployed applications. Dynamic programming (e.g., [57]), genetic algorithms (e.g., [7,58]) and deep learning (e.g., [59]) were exploited promisingly in some recent works. Overall, to the best of our knowledge, none of the previous work in the field of application placement included the possibility to look for secure deployments in Cloud-Edge scenarios, based on application requirements and infrastructure capabilities.

When it comes to trust models and trust management [60], other works such as [30,31] employ (weighted) logic programming and consider networks of trust (and their closures) with values in the range $[0, 1]$ to express trust relations. Also [61] relies on logic programming to define a trust framework for role-based access policies. As in SecFog, such relations describe the belief of one stakeholder to trust another, based of the interactions they

previously had. In line with other trust models [25,62,63], we aggregate multiple trust paths. In the context of ad-hoc networks, much work was done to devise certification based trust models and protocols to spread trust opinions at runtime [64]. Recently, certification-based schemes were proposed also for the Cloud scenario as in the works by Anisetti et al. [65,66].

Trust and customer feedback have been employed in the definition of some Cloud service selection approaches. For instance, Ding et al. [67] proposed a ranking prediction method for personalised Cloud service selection, which takes into account the customer's attitude and expectation towards quality of service, and exploits collaborative filtering techniques by calculating similarities between customers. Qu et al. [68] proposed a context-aware and credible Cloud service selection mechanism based on aggregating subjective assessments extracted from ordinary Cloud consumers and objective assessments from quantitative performance testing parties. Tang et al. [26] later proposed a sophisticated trust evaluation method for Cloud service selection, where *objective* trust assessments (based on QoS monitoring) and *subjective* trust assessments (based on user feedback ratings) of Cloud services are suitably combined.

7. Concluding remarks

In this paper, we proposed a declarative methodology, SecFog, which can be used to quantitatively assess the security level of multi-service application deployments to Cloud-Edge infrastructures. With a prototype implementation in ProbLog, we have shown how SecFog helps application operators in determining secure deployments based on specific application requirements, available infrastructure capabilities, and considering trust degrees in different Edge and Cloud providers.

To the best of our knowledge, SecFog constitutes a first well-founded, efficient and explainable effort towards such direction. The well-foundedness and efficiency of SecFog are guaranteed by the state-of-the-art resolution algorithms implemented within the ProbLog engine. The possibility of explaining the obtained security assessment also derives from ProbLog functionalities that allow the users to obtain graphical ground programs and proofs for the results of their queries. The SecFog prototype can be fruitfully used with other tools for application deployment so to identify suitable trade-offs among the estimated security level and other deployment performance indicators (e.g., QoS-assurance, resource usage, monthly cost, energy consumption), as we have shown with our prototype FogTorchII.

As future work, we plan to:

- prototype a language-based approach and a GUI to ease the declaration of application security requirements (as in [69]) and to provide a user-friendly view of the recommended deployment(s), by suitably highlighting how the application security requirements are satisfied,
- extend such GUI with a visual explanation of the reasons why a given deployment is *not* recommended by SecFog, and
- engineer and integrate SecFog with FogTorchII and show their applicability to actual use cases.

We also intend to enrich the current application model of SecFog so to be able to analyse the security of (probabilistic) information flows among the constituent services, by also defining pre-defined patterns (along the lines of [70]).

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work has been partly supported by the project “DE-CLWARE: Declarative methodologies of application design and deployment” (PRA_2018_66) funded by the University of Pisa, Italy and by the project “GIÒ: a Fog computing testbed for research & education”, funded by the Department of Computer Science, University of Pisa, Italy.

References

- [1] I. Fajjari, F. Tobagi, Y. Takahashi, Cloud Edge Computing in the IoT, Springer, 2018.
- [2] B. Familiar, IoT and microservices, in: Microservices, IoT, and Azure, Springer, 2015, pp. 133–163.
- [3] A. Brogi, S. Forti, A. Ibrahim, Predictive Analysis to Support Fog Application Deployment, in: Rajkumar. Buyya, Satish.N. Srirama (Eds.), Fog and Edge Computing: Principles and Paradigms, Wiley, 2019, pp. 191–222.
- [4] A. Brogi, S. Forti, A. Ibrahim, L. Rinaldi, Bonsai in the Fog: An Active Learning Lab with Fog Computing, in: Fog and Mobile Edge Computing (FMEC), 2018 Third International Conference on, IEEE, 2018, pp. 79–86.
- [5] H. Gupta, A. Vahid Dastjerdi, S.K. Ghosh, R. Buyya, iFogSim: A toolkit for modeling and simulation of resource management techniques in the Internet of Things, Edge and Fog computing environments, Softw. - Pract. Exp. 47 (9) (2017) 1275–1296.
- [6] C. Guerrero, I. Lera, C. Juiz, A lightweight decentralized service placement policy for performance optimization in fog computing, J. Ambient Intell. Humaniz. Comput. (2018).
- [7] O. Skarlat, M. Nardelli, S. Schulte, S. Dustdar, Towards QoS-aware fog service placement, in: 2017 IEEE 1st International Conference on Fog and Edge Computing, IC FEC, 2017, pp. 89–96.
- [8] A. Brogi, S. Forti, QoS-Aware Deployment of IoT Applications Through the Fog, IEEE Internet Things J. 4 (5) (2017) 1185–1192.
- [9] A. Brogi, S. Forti, A. Ibrahim, How to best deploy your Fog applications, probably, in: O. Rana and R. Buyya and A. Anjum (Eds.), Proceedings of 1st IEEE Int. Conference on Fog and Edge Computing, 2017.
- [10] S. Forti, A. Ibrahim, A. Brogi, Mimicking FogDirector Application Management, Software-Intensive Cyber-Physical Systems 34 (2–3) (2019) 151–161.
- [11] A. Brogi, S. Forti, C. Guerrero, I. Lera, How to Place Your Apps in the Fog - State of the Art and Open Challenges, 2019, arXiv preprint arXiv: 1901.05717.
- [12] L. Viganò, D. Magazzeni, Explainable security, 2018, arXiv preprint arXiv: 1807.04178.
- [13] J. Ni, K. Zhang, X. Lin, X. Shen, Securing fog computing for internet of things applications: Challenges and solutions, IEEE Commun. Surv. Tutor. (2017).
- [14] M. Ambrosin, M. Conti, A. Ibrahim, G. Neven, A.-R. Sadeghi, M. Schunter, SANA: secure and scalable aggregate network attestation, in: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, ACM, 2016, pp. 731–742.
- [15] N. Torkzaban, C. Papagianni, J.S. Baras, Trust-aware service chain embedding, in: Proceedings of the 6th International Conference on Software Defined Systems, SDS, 2019.
- [16] P. Garcia Lopez, A. Montresor, D. Epema, A. Datta, T. Higashino, A. Iammitchi, M. Barcellos, P. Felber, E. Riviere, Edge-centric computing: Vision and challenges, ACM SIGCOMM Comput. Commun. Rev. 45 (5) (2015) 37–42.
- [17] L. De Raedt, A. Kimmig, H. Toivonen, ProbLog: A probabilistic prolog and its application in link discovery, in: Proceedings of the 20th International Joint Conference on Artificial Intelligence, 2007, pp. 2468–2473.
- [18] L. De Raedt, A. Kimmig, Probabilistic (logic) programming concepts, Mach. Learn. 100 (1) (2015) 5–47.
- [19] J.-P. Arcangeli, R. Boujbel, S. Leriche, Automatic deployment of distributed software systems: Definitions and state of the art, J. Syst. Softw. 103 (2015) 198–218.
- [20] OpenFog consortium, 2019, <http://www.openfogconsortium.org/>.
- [21] EU cloud SLA standardisation guidelines, 2014.
- [22] H. Mezni, M. Sellami, J. Kouki, Security-aware SaaS placement using swarm intelligence, J. Softw.: Evol. Process (2018).
- [23] M. Mukherjee, R. Matam, L. Shu, L. Maglaras, M.A. Ferrag, N. Choudhury, V. Kumar, Security and privacy in fog computing: Challenges, IEEE Access 5 (2017) 19293–19304.
- [24] M.A. Rodríguez, M.J. Egenhofer, Determining semantic similarity among entity classes from different ontologies, IEEE Trans. Knowl. Data Eng. 15 (2) (2003) 442–456.

- [25] G. Theodorakopoulos, J.S. Baras, On trust models and trust evaluation metrics for ad-hoc networks, *IEEE J. Sel. Areas Commun.* 24 (LCA-ARTICLE-2007-016) (2006) 318–328.
- [26] M. Tang, X. Dai, J. Liu, J. Chen, Towards a trust evaluation middleware for cloud service selection, *Future Gener. Comput. Syst.* 74 (2017) 302–312.
- [27] A. Brogi, S. Forti, A. Ibrahim, Optimising QoS-assurance, Resource Usage and Cost of Fog Application Deployments, in: V. Muñoz, D. Ferguson, M. Helfert, C. Pahl (Eds.), *Cloud Computing and Services Science (CLOSER 2018) Revised Selected Papers*, in: *Communications in Computer and Information Science (CCIS)*, 1073, Springer, 2019, pp. 168–189.
- [28] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, "O'Reilly Media, Inc.", 2015.
- [29] Z. Yan, et al., *Trust Management for Mobile Computing Platforms*, Helsinki University of Technology, 2007.
- [30] S. Bistarelli, F. Martinelli, F. Santini, Weighted datalog and levels of trust, in: *Availability, Reliability and Security*, 2008. ARES 08. Third International Conference on, IEEE, 2008, pp. 1128–1134.
- [31] S. Bistarelli, S.N. Foley, B. O'Sullivan, F. Santini, Semiring-based frameworks for trust propagation in small-world networks and coalition formation criteria, *Secur. Commun. Netw.* 3 (6) (2010) 595–610.
- [32] P. Gao, H. Miao, J.S. Baras, J. Golbeck, Star: Semiring trust inference for trust-aware social recommenders, in: *Proceedings of the 10th ACM Conference on Recommender Systems*, ACM, 2016, pp. 301–308.
- [33] A. Kimmig, G. Van den Broeck, L. De Raedt, An algebraic Prolog for reasoning about possible worlds, in: *Twenty-Fifth AAAI Conference on Artificial Intelligence*, 2011.
- [34] A. Kaur, M. Singh, P. Singh, et al., A taxonomy, survey on placement of virtual machines in cloud, in: *2017 International Conference on Energy, Communication, Data Analytics and Soft Computing (ICECDS)*, IEEE, 2017, pp. 2054–2058.
- [35] S. Schoenen, Z.Á. Mann, A. Metzger, Using risk patterns to identify violations of data protection policies in cloud systems, in: *International Conference on Service-Oriented Computing*, Springer, 2017, pp. 296–307.
- [36] A.A. Nacer, E. Goettelmann, S. Youcef, A. Tari, C. Godart, Obfuscating a business process by splitting its logic with fake fragments for securing a multi-cloud deployment, in: *Services (SERVICES)*, 2016 IEEE World Congress on, IEEE, 2016, pp. 18–25.
- [37] E. Goettelmann, K. Dahman, B. Gateau, E. Dubois, C. Godart, A security risk assessment model for business process deployment in the cloud, in: *Services Computing (SCC)*, 2014 IEEE International Conference on, IEEE, 2014, pp. 307–314.
- [38] Z. Wen, J. Cała, P. Watson, A. Romanovsky, Cost effective, reliable and secure workflow deployment over federated clouds, *IEEE Trans. Serv. Comput.* 10 (6) (2017) 929–941.
- [39] J. Luna, A. Taha, R. Trapero, N. Suri, Quantitative reasoning about cloud security using service level agreements, *IEEE Trans. Cloud Comput.* 5 (3) (2017) 457–471.
- [40] P. Varshney, Y. Simmhan, Demystifying fog computing: Characterizing architectures, applications and abstractions, in: *2017 IEEE 1st International Conference on Fog and Edge Computing, ICFC*, 2017, pp. 115–124.
- [41] Z. Wen, R. Yang, P. Garraghan, T. Lin, J. Xu, M. Rovatsos, Fog orchestration for internet of things services, *IEEE Internet Comput.* 21 (2) (2017) 16–24.
- [42] R. Mahmud, K. Ramamohanarao, R. Buyya, Latency-aware application module management for fog computing environments, *ACM Trans. Internet Technol. (TOIT)* (2018).
- [43] S. Wang, M. Zafer, K.K. Leung, Online placement of multi-component applications in edge computing environments, *IEEE Access* 5 (2017) 2514–2533.
- [44] A. Brogi, S. Forti, A. Ibrahim, Deploying Fog Applications: How Much Does It Cost, By the Way? in: *Proceedings of the 8th International Conference on Cloud Computing and Services Science, SciTePress*, 2018, pp. 68–77.
- [45] H.J. Hong, P.H. Tsai, C.H. Hsu, Dynamic module deployment in a fog computing platform, in: *2016 18th Asia-Pacific Network Operations and Management Symposium, APNOMS*, 2016, pp. 1–6.
- [46] M. Taneja, A. Davy, Resource aware placement of IoT application modules in Fog-Cloud Computing Paradigm, in: *2017 IFIP/IEEE Symposium on Integrated Network and Service Management, IM*, 2017, pp. 1222–1228.
- [47] A.P. Hamid Reza Arkian, Abolfazl. Diyanat, MIST: Fog-based data analytics scheme with cost-efficient resource provisioning for IoT crowdsensing applications, *J. Netw. Comput. Appl.* 82 (2017) 152–165.
- [48] L. Yang, J. Cao, G. Liang, X. Han, Cost aware service placement and load dispatching in mobile cloud systems, *IEEE Trans. Comput.* 65 (5) (2016) 1440–1452.
- [49] L. Gu, D. Zeng, S. Guo, A. Barnawi, Y. Xiang, Cost efficient resource management in fog computing supported medical cyber-physical system, *IEEE Trans. Emerg. Top. Comput.* 5 (1) (2017) 108–119.
- [50] D. Zeng, L. Gu, S. Guo, Z. Cheng, S. Yu, Joint optimization of task scheduling and image placement in fog computing supported software-defined embedded system, *IEEE Trans. Comput.* 65 (12) (2016) 3702–3712.
- [51] V.B.C. Souza, W. Ramirez, X. Masip-Bruin, E. Marn-Tordera, G. Ren, G. Tashakor, Handling service allocation in combined Fog-cloud scenarios, in: *2016 IEEE International Conference on Communications, ICC*, 2016, pp. 1–5.
- [52] M. Barcelo, A. Correa, J. Llorca, A.M. Tulino, J.L. Vicario, A. Morell, IoT-cloud service optimization in next generation smart environments, *IEEE J. Sel. Areas Commun.* 34 (12) (2016) 4077–4090.
- [53] Z. Huang, K.-J. Lin, S.-Y. Yu, J.Y.-j. Hsu, Co-locating services in IoT systems to minimize the communication energy cost, *J. Innov. Digit. Ecosyst.* 1 (1–2) (2014) 47–57.
- [54] R. Mahmud, S.N. Srirama, K. Ramamohanarao, R. Buyya, Quality of experience (QoE)-aware placement of applications in Fog computing environments, *J. Parallel Distrib. Comput.* (2018).
- [55] S. Venticinque, A. Amato, A methodology for deployment of IoT application in fog, *J. Ambient Intell. Humaniz. Comput.* (2018).
- [56] V. Cardellini, V. Grassi, F. Lo Presti, M. Nardelli, Optimal operator placement for distributed stream processing applications, in: *Proceedings of the 10th ACM International Conference on Distributed and Event-Based Systems, DEBS '16*, ACM, New York, NY, USA, 2016, pp. 69–80.
- [57] D. Rahbari, M. Nickray, Scheduling of fog networks with optimized knapsack by symbiotic organisms search, in: *2017 21st Conference on Open Innovations Association, FRUCT*, 2017, pp. 278–283.
- [58] A. Brogi, S. Forti, C. Guerrero, I. Lera, Meet Genetic Algorithms in Monte Carlo: Optimised Placement of Multi-Service Applications in the Fog, in: *Proceedings of the 3rd IEEE International Conference on Edge Computing (EDGE 2019)*, 2019, pp. 13–17.
- [59] Z. Tang, X. Zhou, F. Zhang, W. Jia, W. Zhao, Migration modeling and learning algorithms for containers in fog computing, *IEEE Trans. Serv. Comput.* (2018).
- [60] C.A. Ardagna, E. Damiani, S. De Capitani di Vimercati, S. Foresti, P. Samarati, Trust management, in: M. Petković, W. Jonker (Eds.), *Security, Privacy, and Trust in Modern Data Management*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2007, pp. 103–117.
- [61] N. Li, J.C. Mitchell, W.H. Winsborough, Design of a role-based trust-management framework, in: *Security and Privacy*, 2002. Proceedings. 2002 IEEE Symposium on, IEEE, 2002, pp. 114–130.
- [62] C.-N. Ziegler, G. Lausen, Propagation models for trust and distrust in social networks, *Inf. Syst. Front.* 7 (4–5) (2005) 337–358.
- [63] A. Twigg, N. Dimmock, Attack-resistance of computational trust models, in: *Enabling Technologies: Infrastructure for Collaborative Enterprises*, 2003. WET ICE 2003. Proceedings. Twelfth IEEE International Workshops on, IEEE, 2003, pp. 275–280.
- [64] M. Omar, Y. Challal, A. Bouabdallah, Certification-based trust models in mobile ad hoc networks: A survey and taxonomy, *J. Netw. Comput. Appl.* 35 (1) (2012) 268–286.
- [65] M. Anisetti, C.A. Ardagna, E. Damiani, A certification-based trust model for autonomic cloud computing systems, in: *Cloud and Autonomic Computing (ICCAC)*, 2014 International Conference on, IEEE, 2014, pp. 212–219.
- [66] M. Anisetti, C. Ardagna, E. Damiani, F. Gaudenzi, A semi-automatic and trustworthy scheme for continuous cloud service certification, *IEEE Trans. Serv. Comput.* (2017).
- [67] S. Ding, Z. Wang, D. Wu, D.L. Olson, Utilizing customer satisfaction in ranking prediction for personalized cloud service selection, *Decis. Support Syst.* 93 (2017) 1–10.
- [68] L. Qu, Y. Wang, M.A. Orgun, L. Liu, H. Liu, A. Bouguettaya, CCloud: Context-aware and credible cloud service selection based on subjective assessment and objective assessment, *IEEE Trans. Serv. Comput.* 8 (3) (2015) 369–383.
- [69] S. De Capitani Di Vimercati, S. Foresti, G. Livraga, V. Piuri, P. Samarati, IEEE Transactions on Services Computing, Supporting User Requirements and Preferences in Cloud Plan Selection, In press.
- [70] P. Tsankov, Security analysis of smart contracts in datalog, in: *International Symposium on Leveraging Applications of Formal Methods*, Springer, 2018, pp. 316–322.



Stefano Forti received the MSc degree in computer science and networking (2016) jointly from the Sant'Anna School of Advanced Studies, Pisa, Italy and the University of Pisa, Italy, where he is currently pursuing the PhD degree at the Department of Computer Science. His current research interests include fog, cloud, and service-oriented computing and formal methods and algorithms.



Gian-Luigi Ferrari received the PhD degree in computer science from the University of Pisa, where he is Full Professor at the Department of Computer Science since 2011. His present research interests include formal methods for software engineering, program analysis, security for service-oriented applications and programming languages design for pervasive software systems.



Antonio Brogi is full professor at the Department of Computer Science, University of Pisa (Italy) since 2004. He holds a PhD in Computer Science (1993) from the University of Pisa. His research interests include service-oriented, cloud-based and fog computing, coordination and adaptation of software elements, and formal methods. He has published the results of his research in over 150 papers in international journals and conferences.